Cryptography
Project: CryptoBench

Martin Harrigan, ✉ martin.harrigan@setu.ie

The deadline for the project is 23:59 on 5th December 2025. The project is worth 30 % **of your final grade**. You must submit a report containing a link to the code and data on Blackboard. You have unlimited submission attempts but only your last submission will be marked. Late submissions will not be accepted. Keep your submission receipt.

For the project you are required to benchmark the implementations of a selection of symmetric and asymmetric cryptography algorithms using Python's `cryptography` package and produce a report detailing your results. Specifically, you are required to benchmark five processes: key pair generation, symmetric and asymmetric encryption, symmetric and asymmetric decryption, digital signing, and signature verification. For each process you are required to compare the AES, ChaCha20, RSA, DSA, and ECC implementations, where applicable. I outline the requirements for each of the five processes below.

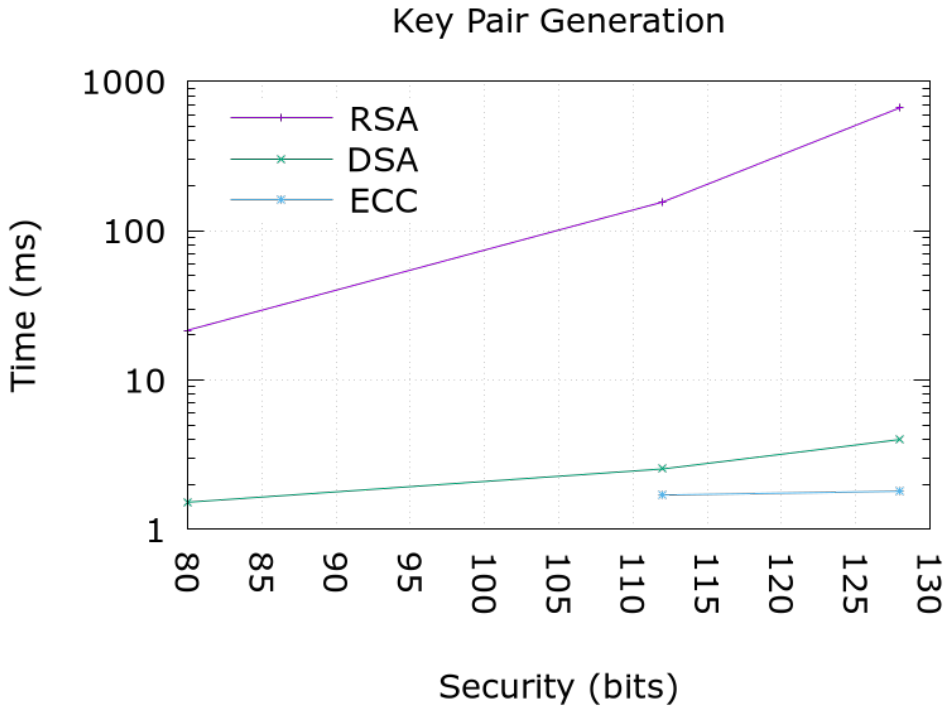


Figure 1: Key pair generation using the RSA, DSA, and ECC algorithms.

Key Pair Generation You are required to generate key pairs using the RSA, DSA, and ECC algorithms. For each algorithm you are required to generate key pairs that provide 80 bit, 112 bit, 128 bit, 192 bit and 256 bit security, where possible. Report your results in a table and compare the generation times using a plot similar to Fig. 1: the x-axis shows the security level in bits and the y-axis shows the running-time in milliseconds; each coloured line corresponds to a different algorithm. For each data point, you should repeat the process a fixed number of times and compute the average of all but the first. For example, suppose I try to compute a 1024 bit RSA key pair and the times reported are 25.10 ms, 19.84 ms and 18.90 ms. I should ignore the first time and compute the average of the other two (19.37 ms). When performing your benchmarks try to compute the average over a large number of runs, e.g., 10 or 100 times where possible.

Symmetric and Asymmetric Encryption You are required to encrypt a plaintext message using the AES, ChaCha20, and RSA algorithms. The plaintext should be randomly generated and $10\,\mathrm{KiB}$. You are required to use the various key sizes for AES ($128\,\mathrm{bit}$, $192\,\mathrm{bit}$ and $256\,\mathrm{bit}$) and the various key pairs generated above ($80\,\mathrm{bit}$, $112\,\mathrm{bit}$, $128\,\mathrm{bit}$, $192\,\mathrm{bit}$ and $256\,\mathrm{bit}$ security) for RSA. For each data point, you should repeat the process and compute an average as above. Report your results in a table and compare the times using a plot.

Symmetric and Asymmetric Decryption You are required to decrypt ciphertexts, produced using the AES, ChaCha20, and RSA algorithm as described above, across the various key and key pair sizes. For each data point, you should repeat the process and compute an average as above. Report your results in a table and compare the times using a plot.

Digital Signing You are required to digitally sign a message using the RSA, DSA and ECC algorithms. The message should be randomly generated and $10\,\mathrm{KiB}$. You should sign the hash of the message rather than the message directly. For each algorithm you are required to use the various key pairs generated above ($80\,\mathrm{bit}$, $112\,\mathrm{bit}$, $128\,\mathrm{bit}$, $192\,\mathrm{bit}$ and $256\,\mathrm{bit}$ security), where possible. Specify which hash function you used in each case. For each data point, you should repeat the process and compute an average as above. Report your results in a table and compare the times using a plot.

Signature Verification You are required to verify signatures, produced using the RSA, DSA and ECC algorithms as described above, across the various key pair sizes. For each data point, you should repeat the process and compute an average as above. Report your results in a table and compare the times using a plot.

Format of your Report Your report should have the following format:

1. Introduction and Experimental Setup: What are the specifications of the machine you are using? What operating system are your using? What version of Python and what version of the `cryptography` package are you using? There should be enough detail for a third-party to replicate your setup.
2. Results
   (a) Results for Key Pair Generation
   (b) Results for Symmetric and Asymmetric Encryption
   (c) Results for Symmetric and Asymmetric Decryption
   (d) Results for Digital Signing
   (e) Results for Signature Verification
3. Conclusion: Were your results inline with your expectations? Did you observe any unexpected results? Can you explain those outliers?

You should include a link to your Python code and the resulting data in your report. The code should be written so that it can be executed in one shot to reproduce all of the results. The contents of the report are more important than a word count. Did you make all of the required comparisons? Are your plots clear and legible? Typically, a report will need 500 to 1000 words to document everything.

Reminder "All work submitted by learners for assessment purposes, or for written or oral publication, must be their own work. Where this is informed by the work of others, the source shall be properly attributed and referenced." (See the South East Technological University (SETU) Full-Time Student Handbook, including Academic Regulations.)

Frequently Asked Questions

Where can I put the code and data that I link to from the report?
You can share your code and data using a GitHub repository, a GitLab repository, your Microsoft OneDrive account, or by uploading it to Blackboard. If you want to make your GitHub repository private, you should add me as a collaborator (`https://github.com/setumartin`). If you want to make your GitLab repository private, you should add me as a collaborator (`https://gitlab.com/users/setumartin`). You should not make any changes to the linked code or data beyond the project deadline.

Are there are requirements on the structure or layout of the code?

Content	Marks (100)
Introduction and Experimental Setup: *Does the report adequately describe the machine specification, specify the versions of Python and the* `cryptography` *package, and include a link to the code?*	12.5
Results for Key Pair Generation: *Does the report include data and plots for key pair generation using the RSA, DSA, and ECC algorithms.*	15
Results for Symmetric and Asymmetric Encryption: *Does the report include data and plots for symmetric encryption using the AES and ChaCha20 algorithms and for asymmetric encryption using the RSA algorithm.*	15
Results for Symmetric and Asymmetric Decryption: *Does the report include data and plots for symmetric decryption using the AES and ChaCHa20 algorithms and for asymmetric decryption using the RSA algorithm.*	15
Results for Digital Signing: *Does the report include data and plots for digital signing using the RSA, DSA, and ECC algorithms?*	15
Results for Signature Verification: *Does the report include data and plots for signature verification using the RSA, DSA, and ECC algorithms?*	15
Conclusion: *Does the report discuss the results, compare the results with your expectations, and identify any anomalies?*	12.5

Table 1: The Marking Scheme.

The only requirement, stated above, is that the code should be written so that it can be executed in one shot to reproduce all of the results. In other words, running `python my_project.py` should produce all of the data needed for your plots.

What tool should I use to generate the plots, similar to the plot in Fig. 1?
You can use any plotting or charting tool, e.g., Microsoft Excel, Matplotlib, or Gnuplot.

What is the marking scheme?
The marking scheme is provided in Table 1.